

Laboratorio 4

Anna Carelli (1029621) e Ilaria Consoli (944548)

Introduzione

Il laboratorio implementa la pipeline di clustering e topic modeling su un dataset di articoli medici presi dalla raccolta PubMed online, utilizzando embeddings neurali, riduzione dimensionale e algoritmi di clustering denso. Oltre all'implementazione è richiesta un'analisi sulla configurazione migliore al variare di:

- Modello di embedding
- Parametri UMAP (*n_components*, *n_neighbors*)
- Parametri HDBSCAN (*min_cluster_size*)

Strumenti utilizzati

Il codice è stato implementato con le seguenti librerie:

- *pickle*: per salvare e caricare le liste di dati in file binari .pkl.
- *time*: per calcolare e stampare il tempo di esecuzione totale della pipeline.
- *umap*: per la riduzione della dimensionalità vettoriale degli embedding testuali.
- *hdbscan*: per effettuare il clustering spaziale basato sulla densità (raggruppamento dei dati ridotti).
- *bertopic*: per gestire l'intero processo di estrazione e modellazione dei topic.
- *sentence_transformers*: per convertire il testo in embedding vettoriali tramite modelli pre-addestrati.
- *nltk*, *string* e *re*: per il preprocessing del testo
- *Bio* (*Biopython - modulo Entrez*): per interrogare le API del database NCBI ed estrarre gli abstract da PubMed.
- *random*: per campionare casualmente gli ID degli articoli di PubMed e i topic per i documenti rappresentativi.

Implementazione

Si distribuisce tra due file distinti:

- *LAB4_0.py* abbiamo il recupero del corpus testuale da PubMed e preprocessing. Ne risultano due file distinti in formato pickle: uno preprocessed e l'altro raw.
- *LAB4.py* utilizzando i dati pkl, abbiamo la costruzione dell'intera pipeline basata sulle variabili richieste all'utente in input. Comprende anche la visualizzazione e le statistiche dei risultati.

Per l'esecuzione della pipeline nel file LAB4.py, è stato utilizzato l'ambiente Google Colab. Questo perché l'elaborazione di una grande quantità di record risulta troppo pesante per la sola CPU del PC, mentre sulla piattaforma Colab è possibile sfruttare la GPU T4 in cloud offerta dai servizi di Google.

Recupero dati

Il recupero del corpus avviene tramite E-utilities, un insieme di programmi che fungono da interfaccia per i dataset del National Center for Biotechnology Information (NCBI). Il NCBI è un ente americano che si occupa di garantire l'accesso e milioni di dati medici e divulgativi. Tra questi dataset, PubMed è un insieme di articoli scientifici di carattere medico. Il corpus finalizzato al training è stato costituito prendendo prima 5000 abstract per il testing e successivamente 45000 abstract dal dataset. A causa di un limite di 9999 risultati per query con Entrez, il recupero è stato strutturato con diverse richieste divise per anno per ottenere articoli recenti e senza duplicati.

Prima avviene il recupero di tutti gli id tramite la chiamata di `esearch`, inserendo come keyword l'anno di pubblicazione. Il risultato è una lista di 55000 id per avere margine di validità nel caso trovassimo abstract non validi o vuoti.

```
Python
target_k = 45000
valid_abstracts = []
# Usiamo set() per salvare dati unici e controllare i duplicati
seen_abstracts = set()
all_pmids = set()

print("Raccolta di PMID unici... ")
# Facciamo ricerche separate per anno per aggirare il limite di 9999
years = [2025, 2024, 2023, 2022, 2021, 2020, 2019, 2018, 2017, 2016, 2015,
2014, 2013, 2012]
for year in years:
    print(f" -> Cerco articoli pubblicati nel {year}...")
    search_handle = Entrez.esearch(db="pubmed", term=f"{year}[dp]",
retmax=4000)
    search_results = Entrez.read(search_handle)
    search_handle.close()
    ids = search_results.get("IdList", [])
    all_pmids.update(ids)
    # Recuperiamo più id di quelli richiesti per sicurezza
    if len(all_pmids) >= 55000:
        break

# Convertiamo il set in una lista
all_pmids = list(all_pmids)
print(f"Trovati {len(all_pmids)} ID unici. Inizio il download degli
abstract...")
```

Utilizzando la lista di id, eseguiamo dei fetch in batch da 500 al database, controllando con il set `seen_abstract` di non avere duplicati e salvando solo gli abstract validi in `valid_abstract`.

Python

```
# Recupero dei record dalla lista di id in batch di 500 elementi
batch_size = 500
for i in range(0, len(all_pmids), batch_size):
    batch_ids = all_pmids[i:i + batch_size]
    try:
        fetch_handle = Entrez.efetch(db="pubmed", id=",".join(batch_ids),
retmode="xml")
        fetch_data = Entrez.read(fetch_handle)
        fetch_handle.close()
        records = fetch_data.get("PubmedArticle", [])
        batch_abstracts = extract_abstracts(records)

        # Controllo duplicati e inserimento
        for abstract in batch_abstracts:
            if abstract not in seen_abstracts:
                seen_abstracts.add(abstract)
                valid_abstracts.append(abstract)
                if len(valid_abstracts) == target_k:
                    break

        print(f"\t--> Ottenuti {len(valid_abstracts)} abstract VALIDI e UNICI")
        if len(valid_abstracts) == target_k:
            break
    except Exception as e:
        print(f"Errore durante il download del blocco (ID saltati): {e}")
```

Questa funzione è stata utilizzata a supporto del recupero per estrarre gli abstract da ogni record ottenuto.

Python

```
def extract_abstracts(records):
    abstracts = []
    for record in records:
        article = record.get("MedlineCitation", {}).get("Article", {})
        abstract_obj = article.get("Abstract", {})
        abstract_parts = abstract_obj.get("AbstractText", [])

        abstract_text = " ".join(str(part) for part in abstract_parts).strip()
        if abstract_text:
            abstracts.append(abstract_text)

    return abstracts
```

Preprocessing

Sono state svolte, nell'ordine, le seguenti operazioni:

1. Rimozione dei link e degli URL tramite l'uso di espressioni regolari
2. Tokenizzazione per frasi
3. Word Tokenization e Lowercasing
4. Rimozione dei caratteri non alfabetici e della punteggiatura
5. Rimozione delle stopwords, includendo sia quelle standard per la lingua inglese, sia una lista di dominio aggiunta manualmente
6. Lemmatizzazione
7. Riasssemblaggio della lista di token filtrati in un'unica stringa di testo per ciascun documento.

Il codice è riportato di seguito:

Python

```
def preprocess(data):
    results = []
    stopwords_list = stopwords.words('english')
    domain_stopwords = ['paper', 'method', 'propose', 'proposed', 'approach',
                        'model']
    stopwords_list.extend(domain_stopwords)
    lemmatizer = WordNetLemmatizer()

    for record in data:
        record = re.sub(r'http\S+|www\S+|https\S+', '', record,
                        flags=re.MULTILINE)
        sentence = sent_tokenize(record)
        table = str.maketrans('', '', string.punctuation)
        abstract_tokens = []
        for s in sentence:
            lower_tokens = [w.lower() for w in word_tokenize(s)]
            filtered_tokens = [t.translate(table) for t in lower_tokens if
                              t.isalpha()]
            tokens = [word for word in filtered_tokens if word not in
                      stopwords_list]
            tokens_fin = [lemmatizer.lemmatize(word) for word in tokens]
            abstract_tokens.extend(tokens_fin)
        clean_abstract_string = " ".join(abstract_tokens)
        results.append(clean_abstract_string)
    return results
```

Salvataggio in formato pickle

Salviamo in seguito tutti dati in due file diversi. Così potrei sperimentare anche la qualità della pipeline su dati preprocessed o raw.

Python

```
# Saving the data
print("Salvataggio in processed_data45.pkl...")
with open('processed_data45.pkl', 'wb') as f:
    pickle.dump(preprocessed_data, f)
with open('raw_data.pkl', 'wb') as f:
    pickle.dump(raw_data, f)
```

Applicazione della pipeline

Per rendere il codice più leggibile e modularizzato, abbiamo creato 5 principali funzioni per l'implementazione totale del processo. Prima delle chiamate a queste funzioni, viene richiesto all'utente di inserire i parametri per l'esecuzione, fornendo le opzioni possibili:

Python

```
# Ask for the input =====
in_model = input("Insert one of these: \n\t1. all-MiniLM-L6-v2 \n\t2.
all-mpnet-base-v2 \n\t3. pritamdeka/S-PubMedBert-MS-MARCO \n\t4.
thenlper/gte-small \nEmbedding model: ")
in_compon = int(input("Insert one of these: \n\t1. 5 \n\t2. 10 \n\t3. 15
\nUMAP n_components = "))
in_neigh = int(input("Insert one of these: \n\t1. 15 \n\t2. 30 \n\t3. 50
\nUMAP n_neighbors = "))
in_mincl = int(input("Insert one of these: \n\t1. 50 \n\t2. 100 \n\t3. 200
\nHDBSCAN min_cluster_size = "))

# Informative print =====
print("You chose: ")
#print(f"Data from {datas}")
print(f"\t- Model {in_model}")
print(f"\t- UMAP with n_components={in_compon}, n_neighbors={in_neigh}")
print(f"\t- HDBSCAN with min_cluster_size={in_mincl}")

# Data opening =====
with open('processed_data45.pkl', 'rb') as f:
    data = pickle.load(f)
```

Pipeline

L'intero codice sottostante permetta l'esecuzione dell'intera pipeline ed è inserito nella funzione di main dopo a recupero degli input. Implementa i seguenti step:

1. Registrazione del tempo di esecuzione della pipeline intera
2. Embedding dei dati tramite il modello selezionato
3. Riduzione delle dimensioni con *UMAP*
4. Clustering tramite *HDBSCAN*
5. Creazione del modello *BERTopic* con le precedenti variabili
6. Elaborazione dei topic con il modello ottenuto

Python

```
# Execute the pipeline =====
print("Loading the model...")
model = SentenceTransformer(in_model)
# Start the timer
print("Running the pipeline...")
start_time = time.time()
# Embedding con SentenceTransformer -----
    print("\t--> Computing embeddings...")
embeddings = model.encode(data, show_progress_bar=True)
# UMAP reduction -----
print("\t--> Reducing dimensionality with UMAP...")
umap_model = umap.UMAP(n_components=in_compon,
                       min_dist=0.0,
                       metric="cosine",
                       n_neighbors=in_neigh,
                       random_state=42)
reduced_embeddings = umap_model.fit_transform(embeddings)
# HDBSCAN clustering -----
print("\t--> Clustering with HDBSCAN...")
hdbscan_model = HDBSCAN(min_cluster_size=in_mincl,
                        metric="euclidean",
                        cluster_selection_method="eom",
                        prediction_data=True)
clusters = hdbscan_model.fit_predict(reduced_embeddings)
# BERTopic modeling -----
print("\t--> Executing with BERTopic...")
topic_model = BERTopic(embedding_model=model,
                       umap_model=umap_model,
                       hdbscan_model=hdbscan_model,
                       verbose=True)
topics, probs = topic_model.fit_transform(data, embeddings)
# End the timer -----
print("Ending the pipeline...")
end_time = time.time()
execution_time = end_time - start_time
```

Stampa delle statistiche richieste

I seguenti metodi permettono la stampa dei seguenti valori:

1. Numero di topic ottenuti
2. Percentuale di outliers
3. Top 5 parole per i 10 topic più utilizzati
4. Tempo di esecuzione
5. Documenti rappresentativi per 5 topic randomici

Python

```
def print_statistics(topic_model, topics, execution_time):
    print("Statistics =====")
    # 1. Number of topics
    topic_info = topic_model.get_topic_info()
    print(f"\tNumber of topics: {len(topic_info) - 1}")
    # 2. Outliers percentage
    outlier_percentage = (topics.count(-1) / len(topics)) * 100
    print(f"\tOutliers percentage: {outlier_percentage:.2f}%")
    # 3. Top 5 words for 10 most used topics
    print(f"\tTop 5 words from 10 most used topics:")
    print_top_topics(topic_model, topic_info)
    # 4. Execution time
    print(f"\tExecution time: {execution_time:.2f} seconds")
    # 5. Representative documents for 5 topics
    print(f"\tRepresentative documents for 5 topics:")
    print_representative_docs(topic_model, topic_info)
```

Per una modularizzare il codice chiamiamo le funzioni seguenti per le stampe che necessitano di iterare sui topic.

Python

```
def print_top_topics(topic_model, topic_info):
    top_10_topics = topic_info[topic_info['Topic'] != -1].head(10)
    for index, row in top_10_topics.iterrows():
        topic_id = row['Topic']
        doc_count = row['Count']
        top_words = [word_tuple[0] for word_tuple in
            topic_model.get_topic(topic_id)[:5]]
        words_string = ", ".join(top_words)
        print(f"\t\t• Topic {topic_id} ({doc_count} documents):
{words_string}")

def print_representative_docs(topic_model, topic_info):
    random_topics = topic_info[topic_info['Topic'] != -1].sample(5 if
len(topic_info) - 1 > 5 else len(topic_info) - 1,
random_state=42)['Topic'].tolist()
    for topic_id in random_topics:
```

```

print(f"\t\tTopic {topic_id}:")
representative_docs = topic_model.get_representative_docs(topic_id)
for i in range(len(representative_docs)):
    print("\t\t\t" + representative_docs[i])

```

Visualizzazione

Per la visualizzazione dei risultati vengono creati e salvati come file html i grafici:

1. Scatter plot per cluster
2. Barplot per le parole più usate nei topic
3. Dendrogramma gerarchico per relazione tra topic
4. Heatmap per relazioni tra topic

Python

```

def visualize(topic_model, data, reduced_embeddings):
    fig_docs = topic_model.visualize_documents(data,
reduced_embeddings=reduced_embeddings, hide_document_hover=False)
    fig_bar = topic_model.visualize_barchart(top_n_topics=10)
    fig_hier = topic_model.visualize_hierarchy()
    fig_heat = topic_model.visualize_heatmap()

    output_dir = "visualizations"
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
    print(f"Saving visualizations to the '{output_dir}' directory...")
    fig_docs.write_html(os.path.join(output_dir,
"documents_visualization.html"))
    fig_bar.write_html(os.path.join(output_dir, "barchart_visualization.html"))
    fig_hier.write_html(os.path.join(output_dir,
"hierarchy_visualization.html"))
    fig_heat.write_html(os.path.join(output_dir, "heatmap_visualization.html"))

```

Configurazioni

Vediamo l'influenza di ciascun parametro sul risultato dell'esecuzione:

- **Modello di embedding:** definisce la qualità e la complessità spaziale di partenza. Modelli più leggeri creano una struttura più semplice e facile da raggruppare, ottima per argomenti generici. Modelli più grandi o domain-specific colgono sfumature semantiche molto più profonde, ma generano spazi ad altissima dimensionalità.
 - **all-MiniLM-L6-v2:** modello leggero e veloce ottimo per task generici. Produce vettori densi di 384 dimensioni
 - **all-mpnet-base-v2:** è più pesante del precedente ma genera vettori a 768 dimensioni

- **pritamdeka/S-PubMedBert-MS-MARCO**: specializzato nell'elaborazione di testi medici scientifici, genera vettori da 768 dimensioni
- **UMAP**
 - **n_components**: definisce a quante dimensioni UMAP deve ridurre i vettori originali prima di passarli all'algoritmo di clustering. I valori bassi comprimono fortemente i dati e di conseguenza facilitano il lavoro di clustering e riducono i tempi di calcolo. Tuttavia con vettori complessi, si rischia di perdere informazioni.
 - **n_neighbors**: determina la dimensione del contesto che deve osservare per mappare la struttura di ogni singolo documento. I valori bassi lo portano a concentrarsi sulla struttura locale con conseguente numero maggiore di topic. Invece con valori alti si concentra sulla struttura globale.
- **HDBSCAN**
 - **min_cluster_size**: stabilisce la dimensione minima (numero di documenti) che un cluster deve avere per poter essere considerato un topic, altrimenti i suoi documenti vengono scartati come rumore (outliers). Un valore basso permette la sopravvivenza di micro-topic e aumenta la granularità e di conseguenza il rischio di avere topic rumorosi o poco significativi. Al contrario, valori alti portano all'under-segmentation: argomenti diversi vengono fusi insieme pur di raggiungere il numero minimo richiesto.

Testing iniziale

Per decidere i parametri per le 3 configurazioni, abbiamo prima testato le variazioni di ogni parametro. Le nostre osservazioni sono:

- A un *modello* di embedding ad alta dimensionalità va associato *n_components* massimo, così da riuscire a mantenere le sfumature di significato del vettore originale
- Il valore di *n_neighbors* va alzato nel caso di modelli generalisti che quindi faticano a individuare informazioni specifiche di dominio.
- Il *min_cluster_size* deve essere bilanciato rispetto alla densità del modello.

Scelte finali

In base a queste considerazioni, sono state scelte le seguenti configurazioni:

	Embedding	UMAP		HDBSCAN
		n_components	n_neighbors	min_cluster_size
1	all-MiniLM-L6-v2	5	50	50
2	all-mpnet-base-v2	10	50	50
3	S-PubMedBert	15	15	50

Preprocessed vs Raw data

Per avere un confronto sulla differenza tra dati preprocessed e raw, è stata eseguita la pipeline al pari di configurazione sia su dati raw che processed limitati a 5000 record utilizzando la CPU del PC. I risultati ottenuti sono:

	preprocessed	raw
Numero di topic	14	13
Percentuale di outliers	28,69%	37.07%
Tempo computazionale	188.81 seconds	193.62 seconds

I valori riportati non sembrano peggiorati significativamente, tuttavia se osserviamo i documenti rilevanti stampati e i le top 5 keyword per i 10 topic possiamo vedere che la parole chiave si riducono a articoli determinativi, congiunzioni elementi grammaticali che rientrano nelle stopwords. Questo provoca un peggioramento per quanto riguarda la rilevanza e la qualità dei cluster. Infatti nella versione preprocessed, le parole chiave sono più tematiche e coerenti tra loro. Ne riportiamo di seguito i primi 5.

	Raw	Preprocessed
0	(591 documents): and, the, of, to, in	(500 documents): health, care, study, student, among
1	(342 documents): the, and, of, in, to	(404 documents): patient, risk, ci, study, clinical
2	(337 documents): sub, the, sup, of, and	(280 documents): sub, sup, quantum, energy, material
3	(288 documents): and, in, the, of, to	(262 documents): cell, tumor, cancer, immune, therapeutic
4	(268 documents): and, with, the, of, in	(251 documents): cognitive, brain, sleep, study, motor

Dati questi risultati, abbiamo deciso di continuare le analisi su dati preprocessed.

Risultati ottenuti

Di seguito riportiamo le statistiche delle tre configurazioni di pipeline eseguite su dati preprocessed composti da 45000 record.

	1	2	3
Numero di topic	113	114	160
Percentuale di outliers	50.34%	38.91%	50.05%
Tempo computazionale	348.88 sec	1022.57 sec	700.65 sec

Di seguito le stampe delle top 5 parole per i 10 topic più frequenti per ogni esecuzione:

None

1)

- Topic 0 (1364 documents): image, algorithm, mi, mo, data
- Topic 1 (1025 documents): content, acid, antioxidant, fruit, extract
- Topic 2 (560 documents): eye, visual, retinal, corneal, acuity
- Topic 3 (539 documents): cell, cancer, expression, tumor, apoptosis
- Topic 4 (521 documents): student, learning, nurse, nursing, education
- Topic 5 (499 documents): player, athlete, sport, training, muscle
- Topic 6 (496 documents): tumor, gland, rare, lesion, case
- Topic 7 (461 documents): fracture, hip, femoral, fixation, knee
- Topic 8 (405 documents): drug, delivery, release, nanoparticles, formulation
- Topic 9 (390 documents): sub, catalyst, sup, adsorption, carbon

2)

- Topic 0 (3400 documents): sub, sup, material, nanoparticles, property
- Topic 1 (2586 documents): extract, protein, acid, cell, antioxidant
- Topic 2 (1890 documents): data, algorithm, network, image, learning
- Topic 3 (921 documents): tumor, lesion, case, rare, gland
- Topic 4 (685 documents): laparoscopic, patient, bowel, biliary, pancreatic
- Topic 5 (520 documents): sexual, hiv, stigma, transgender, minority
- Topic 6 (512 documents): eye, retinal, visual, corneal, acuity
- Topic 7 (501 documents): player, athlete, sport, training, exercise
- Topic 8 (429 documents): food, eating, obesity, consumption, weight
- Topic 9 (405 documents): pregnancy, maternal, woman, neonatal, fetal

3)

- Topic 0 (811 documents): pandemic, coronavirus, health, lockdown, crisis
- Topic 1 (764 documents): estimator, estimation, data, algorithm, simulation
- Topic 2 (677 documents): parasite, infection, cattle, infected, prevalence
- Topic 3 (561 documents): pain, opioid, chronic, group, analgesic
- Topic 4 (383 documents): laparoscopic, hernia, abdominal, cholecystectomy, biliary
- Topic 5 (373 documents): eye, retinal, visual, macular, acuity
- Topic 6 (340 documents): pharmacist, pharmacy, medication, drug, prescription
- Topic 7 (327 documents): nurse, nursing, care, student, palliative

- Topic 8 (326 documents): cancer, expression, breast, cell, nsc1c
- Topic 9 (287 documents): flour, storage, drying, sensory, starch

Grafici

Si riportano i grafici ottenuti dall'esecuzione in formato html per facilitarne la consultazione interattiva.

Configurazione 1

- Model all-MiniLM-L6-v2
- UMAP with n_components=5, n_neighbors=50
- HDBSCAN with min_cluster_size=50

1.  barchart_1.html
2.  documents_1.html
3.  hierarchy_1.html
4.  heatmap_1.html

Configurazione 2

You chose:

- Model all-mpnet-base-v2
- UMAP with n_components=10, n_neighbors=50
- HDBSCAN with min_cluster_size=50

1.  barchart_2.html
2.  documents_2.html
3.  hierarchy_2.html
4.  heatmap_2.html

Configurazione 3

You chose:

- Model pritamdeka/S-PubMedBert-MS-MARCO
- UMAP with n_components=15, n_neighbors=15
- HDBSCAN with min_cluster_size=50

1.  barchart_3.html
2.  documents_3.html
3.  hierarchy_3.html
4.  heatmap_3.html

Difficoltà riscontrate

Dai test condotti rileviamo diverse difficoltà.

Per quanto riguarda il modello specialistico **PubMedBert**, inizialmente l'algoritmo ha generato un unico Mega Cluster contenente la maggior parte dei documenti.

L'under-clustering è frequente nei dataset medici, dove l'uso di termini comuni (patient, study ...) crea continuità tra argomenti in realtà distinti.

Per risolvere il problema e far emergere i topic più piccoli, è stato impostato ***min_samples*** = 5. Questo valore rende HDBSCAN molto più restrittivo, costringendolo a classificare i documenti generici o di transizione come outliers. Nello specifico, esso definisce quanti punti vicini l'algoritmo deve considerare per decidere se un punto si trova in una regione a bassa o alta densità.

Anche per un modello pesante come **all-mpnet-base-v2** si verifica il fenomeno di under-clustering, questa volta dovuto però al dominio specialistico del corpus. La soluzione è mantenere alta la dimensionalità dell'embedding con `n_components` a 10 e utilizzare più finestra di contesto alzando `n_neighbors` a 50. Così riusciamo a ottenere risultati soddisfacenti e con sufficienti topic che possano rappresentare il dominio frammentato.

Riflessioni

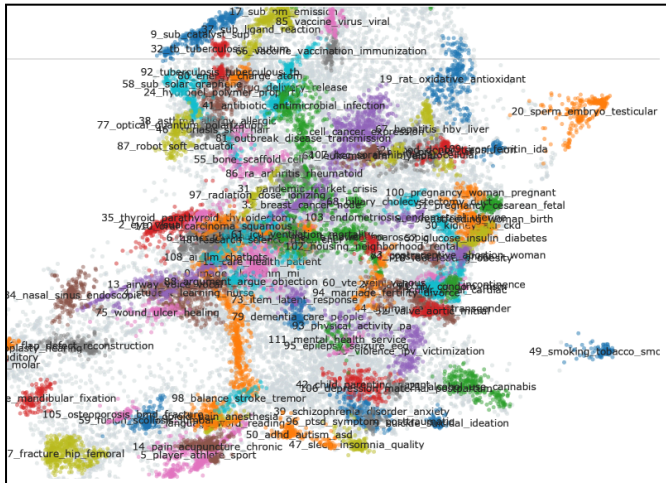
Influenza del modello di embedding

I modelli leggeri come all-MiniLM-L6-v2, generano spazi vettoriali semplici e veloci da elaborare, ottimi per argomenti generici ma inclini a scartare molti documenti come rumore. I modelli iper-specializzati come PubMedBert, pur cogliendo sfumature semantiche profonde, vanno in difficoltà per la presenza di termini comuni ("patient", "study") e creano enormi "mega-cluster" indistinti, obbligando a impostare parametri molto restrittivi per separare i topic.

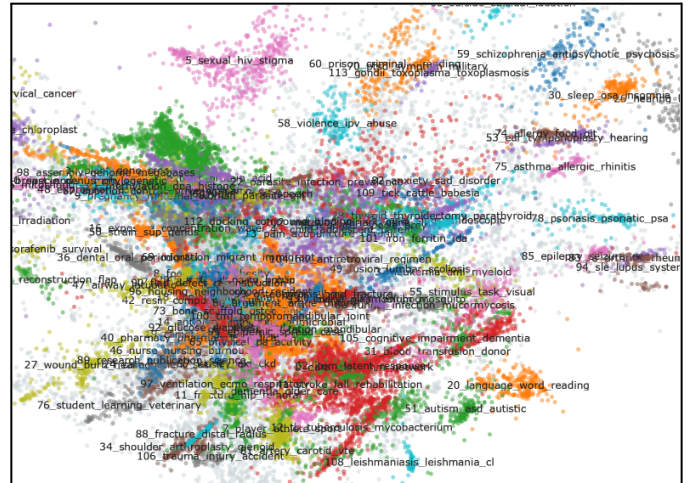
Vale la pena il costo computazionale extra ma dipende dal modello. Il costo extra è pienamente giustificato per modelli come all-mpnet-base-v2: pur triplicando il tempo di esecuzione, offre il miglior compromesso in assoluto. Infatti, pur mantenendo un'ottima ricchezza semantica, riesce ad abbassare drasticamente la percentuale di outliers, inserendo molti più documenti all'interno di cluster ben formati.

Separazione tra cluster

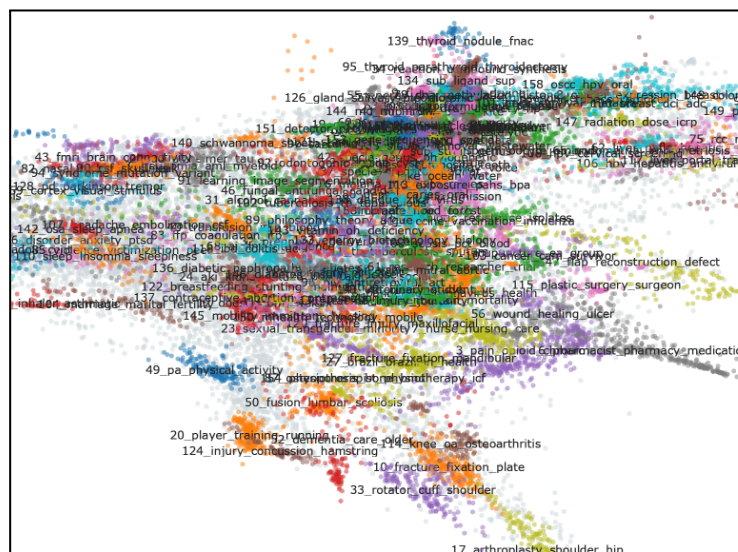
Osservando nel dettaglio il grafico scatter dei documenti, possiamo confrontare la geometria dei cluster delle tre diverse configurazioni. Le immagini mostrate sono un frammento del grafico intero a scopo dimostrativo.



Configurazione 1



Configurazione 2



Configurazione 3

La separazione avviene in modo più evidente nella prima configurazione, che riesce ad espandere lo spazio occupato dai cluster in modo da riuscire a distinguerli. Al contrario, la terza configurazione sovrappone le nubi di documenti, creando un ammasso denso e compresso in cui i confini tra i vari topic diventano invisibili, confermando come un'eccessiva specializzazione del modello, porti a una saturazione dello spazio semantico che compromette la chiarezza della mappa concettuale.

Outlier accettabili

Dai risultati emerge che dal 30% al 50% dei documenti è classificato come rumore, il che non significa necessariamente che i parametri siano troppo stringenti, bensì che il modello sta filtrando con efficacia il rumore semantico del dataset per far emergere solo i nuclei tematici più rilevanti.



La seconda configurazione dimostra l'efficacia di un modello di embedding a più alta dimensionalità, che agisce riducendo la percentuale di outliers al 38.91% e includendo migliaia di documenti altrimenti persi nel rumore. Il trade-off consiste in una forte densità della nube di punti, sintomo del fatto che includere più documenti porta i cluster a toccarsi tra loro, richiedendo un'analisi più attenta per distinguere le sfumature di micro-topic limitrofi.

Coerenza Semantica

Viene determinata osservando sia le principali keywords che i documenti rappresentativi dei topic.

1. Configurazione 1:
 - a. Analisi delle parole chiave: I cluster identificati, risultano coerenti e ben definiti. Il modello è efficace nell'isolamento di macro-aree tematiche. L'unica eccezione è il Topic 0 che contiene termini analitici generici (image, algorithm, mi, mo, data)
 - b. Documenti rappresentativi: I documenti confermano l'omogeneità: i testi all'interno del Topic 4 trattano in modo univoco l'educazione infermieristica e medica come ci confermano le parole chiave ottenute nel punto precedente.
2. Configurazione 2:
 - a. Analisi delle parole chiave: I topic appaiono più ricchi: il Topic 3 (tumor, lesion, case, rare, gland) e il Topic 4 (laparoscopic, patient, bowel, biliary, pancreatic) definiscono ambiti molto specifici, da quello oncologico a quello della chirurgia. Tuttavia, come per la prima configurazione, viene incluso anche il Topic 2 con parole chiave meno tecniche dell'ambito medico.
 - b. Documenti rappresentativi: anche per questo caso, il Topic 4 viene rappresentato in modo coerente da documenti con procedure chirurgiche su pazienti con calcoli biliari o pancreatite.
3. Configurazione 3:

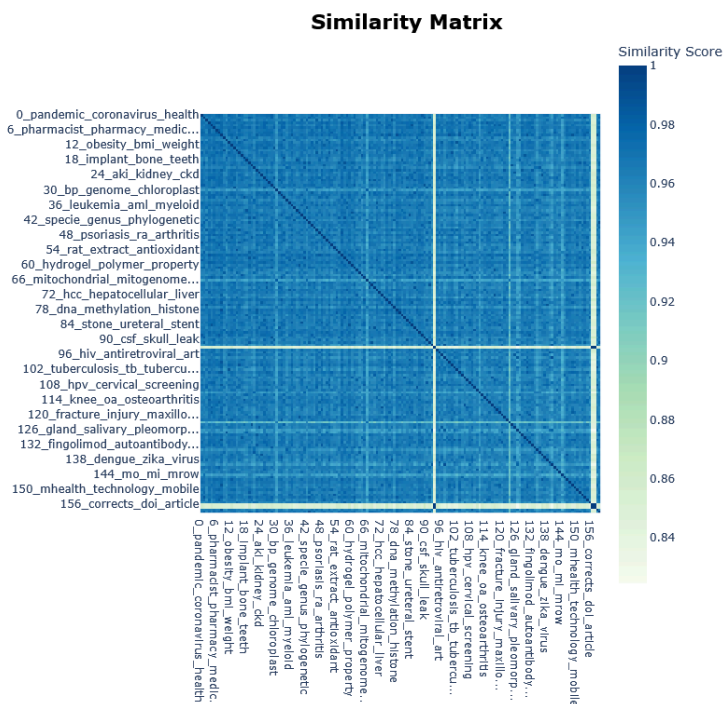
- Analisi delle parole chiave: I topic risultano altamente specifici, senza alcun Topic generico. Si osservano cluster come il Topic 0 (pandemic, coronavirus, lockdown) o il Topic 2 (parasite, infection, cattle, prevalence), che denotano una grande precisione semantica, capace di distinguere contesti che modelli più generici potrebbero confondere.
- Documenti rappresentativi: I documenti del Topic 2 confermano la coerenza di dominio: tutti i testi sono incentrati sulla ricerca di parassiti in ospiti animali e sulle relative metodologie diagnostiche (PCR, esami microscopici).

Over o Under Segmentazione

Individuiamo l'over-segmentation con la presenza di molti topic frammentati e ripetitivi, e al contrario, l'under-segmentation con la presenza di topic troppo ampi, fusi e generici.

Per controllare la presenza di questi fenomeni, analizziamo i grafici Hierarchy e Heatmap: la prima mostra come si uniscono i topic, la seconda mostra il valore numerico di cosine similarity di quella unione.

La configurazione in cui spicca un'evidente over-segmentation è quella di PubMedBert: come possiamo osservare dalla heatmap, i valori minimi partono da una base già alta (circa 0.83) per arrivare a 1.



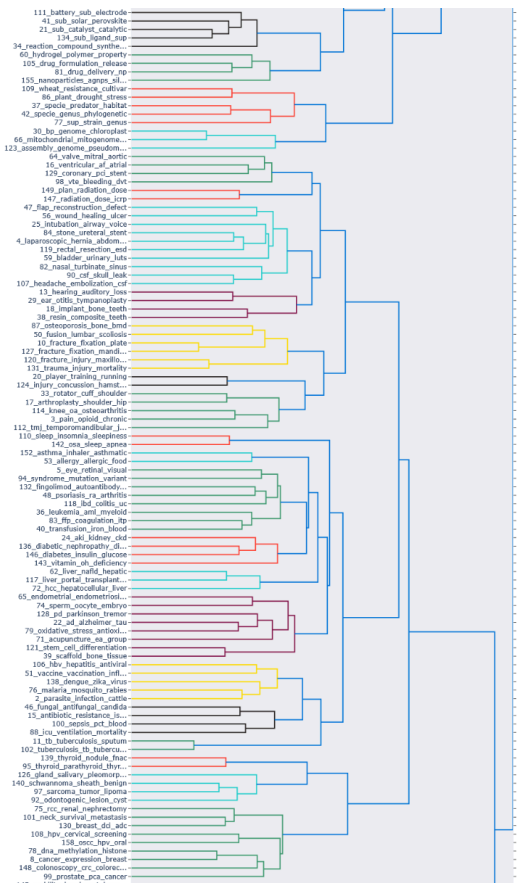
L'intera matrice è dominata in modo omogeneo da un colore blu molto scuro e intenso. Questo indica che la stragrande maggioranza dei topic estratti possiede una cosine similarity elevata l'uno con l'altro. L'algoritmo ha quindi isolato decine di cluster che, dal punto di vista dello spazio semantico, sono quasi indistinguibili, confermando la presenza di troppi topic simili tra loro.

Questa è la prova visiva del compromesso adottato. Per evitare l'under-segmentation iniziale del modello PubMedBert, i parametri sono stati resi molto restrittivi, costringendo il sistema a separare in modo innaturale e creando micro-topic ridondanti.

Anche nella Hierarchy possiamo vedere come le ramificazioni siano fitte nella parte centrale, il che significa che ci sono tanti topic che sono sullo stesso livello e che quindi sono semanticamente vicini.

Il trade-off finale è un'elevata interpretabilità dei singoli micro-cluster a fronte di una struttura ridondante e di un aumento drastico degli outliers.

Per quanto riguarda le altre configurazioni, possiamo dire che sono abbastanza bilanciate e non presentano particolari anomalie.



Interpretabilità dei topic

Un topic si definisce interpretabile quando è possibile assegnargli un significato chiaro, univoco e coerente tramite l'osservazione delle sue parole chiave e dei documenti rappresentativi.

Tra le configurazioni testate, la Configurazione 3 basata su S-PubMedBert è quella che produce i topic dotati della maggiore interpretabilità semantica. Questo perché il modello, essendo pre-addestrato specificamente su testi medici, possiede una sensibilità superiore nel distinguere sfumature concettuali. Tuttavia, l'altra ragione per la sua specificità è l'utilizzo di vincoli molto ristretti che portano a over-segmentation.

Conclusioni

Il laboratorio ha permesso di esplorare l'efficacia di una pipeline di topic modeling applicata a un corpus biomedico di grandi dimensioni, evidenziando come la qualità dei risultati sia il risultato di un delicato equilibrio tra le componenti della pipeline: embedding, riduzione dimensionale e clustering.

In conclusione, la Configurazione 2 si è distinta come la soluzione più equilibrata, offrendo un ottimo compromesso tra capacità di inclusione dei documenti e coerenza semantica dei topic estratti. Il lavoro svolto dimostra che, sebbene non esista una configurazione universale, un'analisi iterativa dei parametri e una valutazione critica della struttura gerarchica dei topic permettono di trasformare un vasto corpus di abstract in una mappa concettuale coerente e navigabile del dominio medico.

Bibliografia

1. Modello di embedding addestrato su PubMed
<https://huggingface.co/pritamdeka/S-PubMedBert-MS-MARCO>
2. PubMed <https://pubmed.ncbi.nlm.nih.gov/>
3. Entrez® Programming Utilities <https://www.ncbi.nlm.nih.gov/books/NBK25501/>